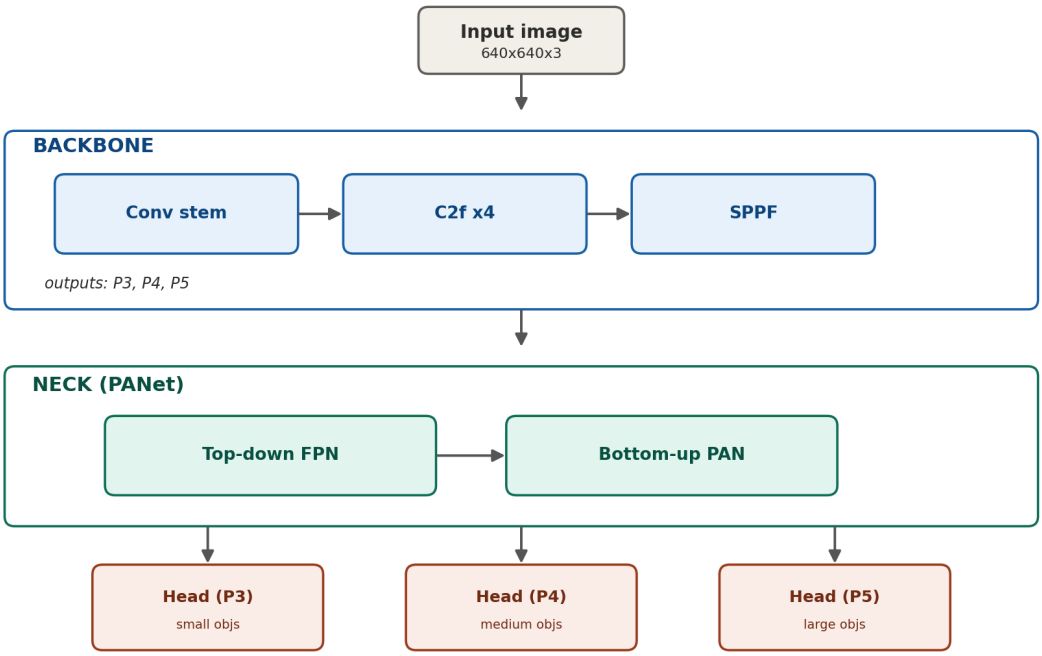
YOLOv8 Architecture — Notes

Short notes with diagrams, deep learning roadmap (object detection)

Full architecture overview



each head: class branch (what) + box branch (where), anchor-free

Flow: Input → Backbone (Conv stem + C2f x4 + SPPF) → Neck (PANet) → Head

1. Input image

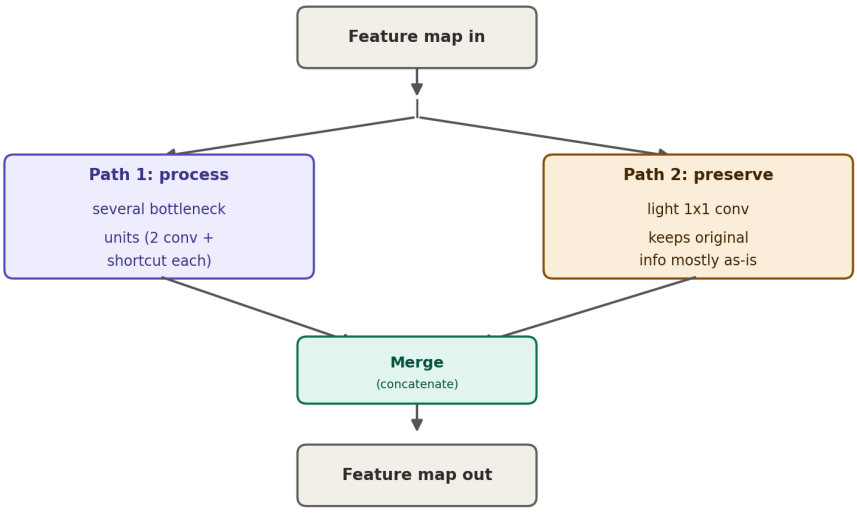
- Image converted to numbers, shape 640x640x3 (H x W x RGB)
- Nothing learned yet, just raw pixel values

2. Conv stem

- First conv layer, stride 2 → jumps 2 pixels at a time

- Shrinks image size, finds basic edges/color changes

3. C2f block (runs 4 times)



Input to a C2f block is a feature map from the previous layer, not the raw image. It splits into 2 paths, does its work, then merges back into one output. Below is the same thing explained slowly in plain words.

Path 1 — the "editor" path

- This path takes the feature map and passes it through several small units called **bottleneck units**, one after another.
- One bottleneck unit = 2 conv layers stacked together + a tiny shortcut inside it.
- Conv layer 1 finds some patterns. Conv layer 2 takes those patterns and finds patterns on top of them — so it goes deeper each time.
- Both conv layers are doing the same kind of job (finding/refining patterns) — layer 2 is just working one level deeper than layer 1, not doing a totally different task.
- After passing through several of these bottleneck units in a row, the features become much more detailed and object-like.

Path 2 — the "drawer copy" path

- This path takes the SAME feature map (not the original raw image, the one entering this C2f block).
- It only passes through one light 1x1 conv — this mainly just adjusts the number of channels, it does not really go hunting for new patterns.
- So this path stays very close to the original input of this block — "untouched" only means untouched compared to Path 1, not untouched since the start of the network.
- Its job is simply to carry the original info forward safely, in case Path 1's deep processing loses something along the way.

Merge

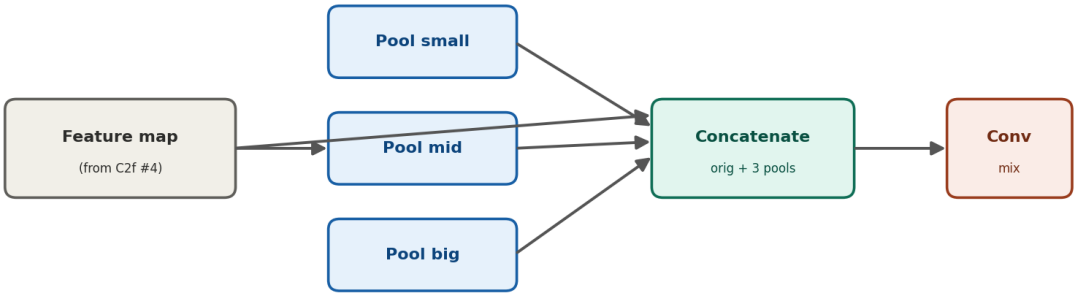
- Path 1's output (deeply refined) and Path 2's output (mostly preserved) are **concatenated** — joined side by side into one bigger feature map.
- This merge happens inside every single C2f block, every time it runs — not just once at the very end of all 4 runs.
- Why keep Path 2 at all: in deep networks the training signal (gradient) can get weak the further back it travels during learning. Path 2 gives it a short, easy route back, so training stays stable.

Simple analogy: 2 copies of a document are made. Copy 1 goes to an editor who keeps refining it, again and again, getting more detailed each time (Path 1). Copy 2 is kept untouched in a drawer as backup (Path 2). At the end, both are stapled together — so we get the refined version AND we never lose the original meaning.

This whole split → process → merge cycle repeats 4 times (C2f #1, #2, #3, #4). Each run shrinks the feature map a bit more and makes the patterns more object-like — first just edges and curves, then

textures, then shapes that resemble parts of a real object.

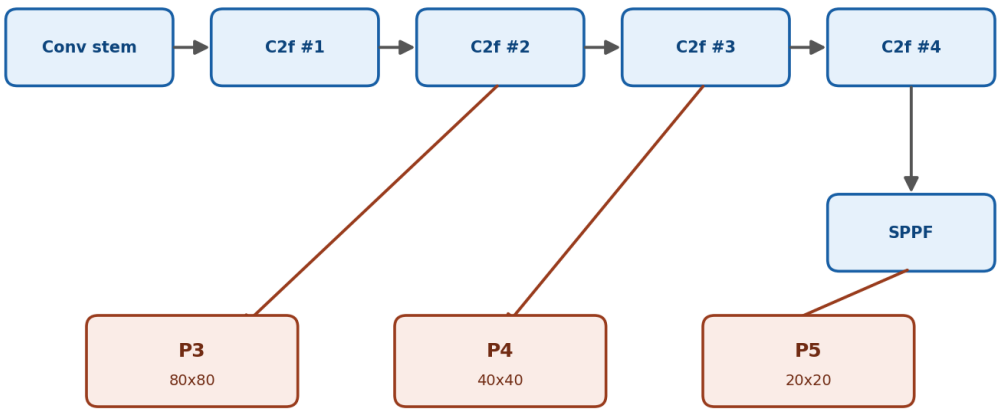
4. SPPF (Spatial Pyramid Pooling – Fast)



small window = fine detail (small objs) | big window = wide context (large objs)

- Input = raw output of C2f block #4
- Problem solved: a single filter can't see small and large objects together
- Max-pooling applied 3 times in a row → small, mid, big effective window
- Small window → fine detail → small objects
- Big window → wide context → large objects
- Concatenate: original feature map + 3 pooled versions (4 things)
- Final conv layer mixes it into one clean output = SPPF output
- Only affects P5's path, not P3/P4

5. P3, P4, P5 — backbone outputs

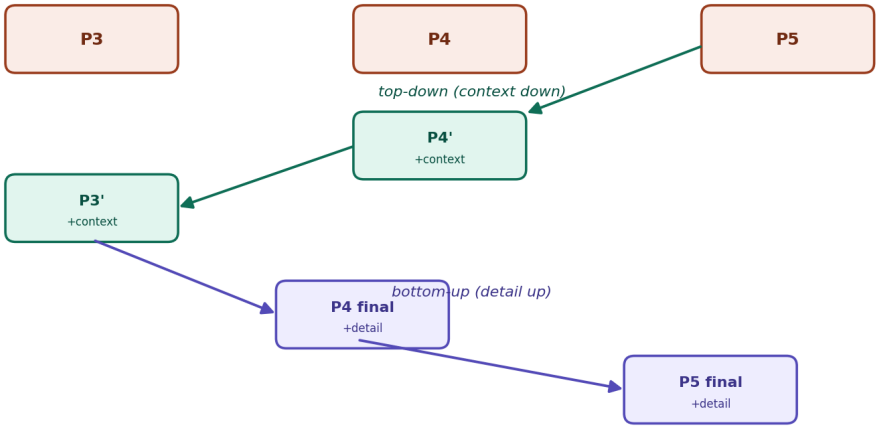


P3 = after C2f #2 | P4 = after C2f #3 | P5 = after C2f #4 + SPPF

Note: different from SPPF's 3 pooling windows. These are 3 separate feature maps from 3 different depths.

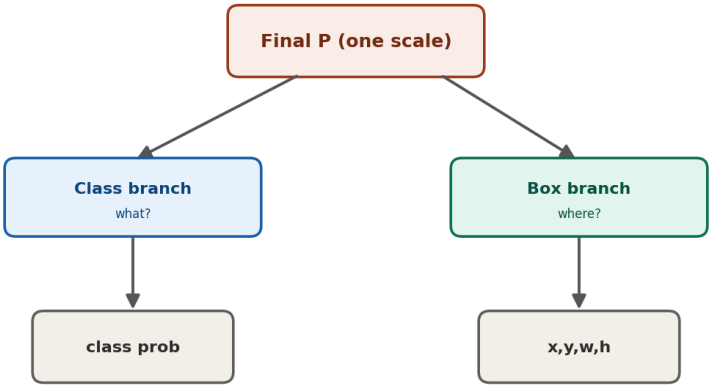
Output	Taken from	Size	Good for
P3	C2f block #2 output	80x80	small objects
P4	C2f block #3 output	40x40	medium objects
P5	C2f #4 + SPPF	20x20	large objects

6. Neck (PANet)



- Input: P3, P4, P5 — currently isolated from each other
- **Step 1 – Top-down (context flows down):**
- P5 → upsample → concat with P4 → improved P4
- improved P4 → upsample → concat with P3 → improved P3
- **Step 2 – Bottom-up (detail flows back up):**
- improved P3 → downsample → concat with improved P4 → final P4
- final P4 → downsample → concat with original P5 → final P5
- Result: all 3 (P3, P4, P5) now have detail + context, not just one

7. Head



- Input: final P3, P4, P5 from neck
- 3 separate heads, one per scale
- Each head decoupled into 2 branches: class branch (what) + box branch (where, x/y/w/h)
- Why separate: what/where need different feature focus, splitting improves both
- Anchor-free: predicts box directly, no pre-defined anchor shapes needed
- Output: class + box per grid cell per scale, cleaned up later with NMS

Quick recap table

Block	One line
Input	raw image as numbers, 640x640x3
Conv stem	stride 2 conv, shrinks + finds edges
C2f x4	split, refine + preserve, merge, repeat 4x
SPPF	3-level pooling, wide context, only for P5
P3/P4/P5	3 depths → small/medium/large objects
Neck top-down	P5 context passed down to P4, P3
Neck bottom-up	P3 detail passed back up to P4, P5
Head	3 heads, class + box branches, anchor-free

Input → Backbone (Conv stem → C2f x4 → SPPF) → Neck (top-down + bottom-up) → Head

---

Next: YOLOv8 Training Pipeline